Compte Rendu : Collecte de Données Web et Optimisation d'un Réseau de Neurones

1. Introduction & Objectifs

Ce compte rendu synthétise le travail de constitution d'un corpus textuel thématique issu de Wikipédia (via API, crawling et scraping) ainsi que l'analyse de l'optimisation des hyperparamètres d'un réseau de neurones multicouche destiné à classifier ces textes par thème (astronomie, cuisine, peinture, football).

---

2. Partie 1 : Collecte de Données (API, Crawling, Scraping)

Méthodologie et Bonnes Pratiques

La collecte de données automatisée sur le web obéit à des règles strictes pour préserver les serveurs cibles :

- **Le respect du `robots.txt` et du `User-Agent` :** L'utilisation d'un `User-Agent` explicite (mentionnant un contact) et la lecture préalable du fichier `robots.txt` permettent d'identifier clairement le script et d'éviter un blocage ou un bannissement (erreurs 403/429).
- **La temporisation (Pops/Pauses) :** L'insertion de pauses (`time.sleep`) entre les requêtes et la gestion des codes d'erreur (tels que 429 ou 503) garantissent une collecte « polie » et stable.

Comparaison des Méthodes de Collecte

1. **L'API (Application Programming Interface) :** C'est la méthode la plus propre et la plus stable. Elle fournit des données déjà structurées (au format JSON), mais nécessite de gérer la pagination (ex: paramètre `cmcontinue` pour l'API Wikipédia).
2. **Le Crawling :** Consiste à parcourir l'arborescence des pages en suivant les liens hypertextes (comme l'exploration récursive des sous-catégories) pour

découvrir des pages lorsque l'on ne dispose pas de liste exhaustive.

3. **Le Scraping :** Consiste à analyser et lire le code source HTML d'une page (grâce à des bibliothèques comme `BeautifulSoup`) pour en extraire des champs précis (titres, paragraphes pertinents en ignorant les infoboxes et tableaux). Il reste indispensable lorsqu'aucune API officielle n'est disponible.

Constitution du Corpus final

- **Volume collecté :** 239 textes répartis sur 4 classes distinctes (astronomie, cuisine, peinture, football), après nettoyage des doublons et des paragraphes vides.
- **Format de sortie :** Un fichier structuré `corpus.csv` contenant les colonnes `classe`, `titre`, `texte` et `lien`, prêt pour l'entraînement de modèles de Deep Learning.

---

3. Partie 2 : Optimisation d'un Réseau de Neurones et Analyse des Hyperparamètres

Rappels Théoriques et Méthodologiques

L'optimisation d'un modèle de Deep Learning repose sur le réglage rigoureux des hyperparamètres (taux d'apprentissage, nombre de neurones, dropout, epochs) en veillant à séparer strictement les rôles des jeux de données :

- **Jeu d'apprentissage :** Pour l'entraînement des poids du modèle.
- **Jeu de validation :** Utilisé uniquement pour comparer les réglages et détecter le surapprentissage.
- **Jeu de test :** Réservé à un **unique** passage final pour obtenir le verdict d'évaluation non biaisé.

Synthèse des Expériences d'Optimisation

Configuration / Réglage	Précision de Validation	Commentaires & Constats
Modèle de référence (64 / 32 neurones)	0,944	Point de départ initial stable.
Taux d'apprentissage (`taux = 0.001` ou	0,944	Un pas de correction adapté permet d'atteindre rapidement la convergence

Configuration / Réglage	Précision de Validation	Commentaires & Constats
`0.01`)		(contrairement à `0.0001` ou `0.1`).
Augmentation de la taille (256 / 128 neurones)	0,944	Élève la complexité mais accentue le risque de surapprentissage (mémorisation).
Ajout de Dropout (`dropout = 0.5`)	0,944	Réduit efficacement la perte de validation finale (`0.16`) en luttant contre la mémorisation excessive.
Modèle Quantifié (TFLite)	0,950	**Gain massif :** Taille du modèle divisée par ~4 (passant de ~509 Ko à 131 Ko) sans perte de précision notable.
Modèle Allégé (8 / 4 neurones)	0,778	Modèle très compact (16 064 paramètres), mais s'accompagne d'une baisse sensible de la performance.

Analyse Statistique et Bruit

- **Incertitude statistique :** Avec de petits jeux de validation, les variations de précision de l'ordre de quelques dixièmes de pourcent (ou de 0,5 point) se situent dans la marge d'incertitude (bruit statistique inhérent à l'initialisation aléatoire des poids via la graine / `random_state`).
- **Gestion du surapprentissage :** L'observation des courbes de perte montre que lorsque le réseau mémorise trop, la perte de validation finit par remonter. Le **dropout** et l'**arrêt précoce** (*early stopping*) constituent des parades efficaces pour stabiliser la généralisation.

---

4. Conclusion

Ce double travail pratique a permis d'illustrer l'ensemble de la chaîne de traitement des données textuelles non structurées : depuis la collecte automatisée respectueuse des standards du web (API, crawling, scraping) jusqu'à la mise en place, l'évaluation rigoureuse et l'allègement (quantification) d'un réseau de neurones en vue d'un déploiement optimisé.

Le 16 octobre 2026